

Treble Lin 語言語法書

目錄

- 詞法結構 (Lexical Structure)
 - 1. 識別字 (Identifiers)
 - 2. 關鍵字 (Keywords)
 - 3. 運算子和標點符號 (Operators and Punctuation)
 - 4. 字面量 (Literals)
 - 5. 註解 (Comments)
- 語法 (Syntax)
 - 1. 語句 (Statements)
 - 1.1 表達式語句 (Expression Statements)
 - 1.2 變數宣告 (Variable Declarations)
 - 1.3 程式碼區塊 (Block Statements)
 - 1.4 條件語句 (Conditional Statements)
 - 1.5 迴圈語句 (Loop Statements)
 - 1.6 函式宣告 (Function Declarations)
 - 1.7 返回語句 (Return Statements)
 - 1.8 錯誤處理 (Error Handling)
 - 2. 表達式 (Expressions)
 - 2.1 賦值表達式 (Assignment Expressions)
 - 2.2 邏輯表達式 (Logical Expressions)
 - 2.3 等值表達式 (Equality Expressions)
 - 2.4 比較表達式 (Comparison Expressions)
 - 2.5 算術表達式 (Arithmetic Expressions)
 - 2.6 一元表達式 (Unary Expressions)
 - 2.7 函式呼叫表達式 (Call Expressions)
 - 2.8 主要表達式 (Primary Expressions)
- 資料型別 (Data Types)
 - 真值性 (Truthiness)
- 執行模型 (Execution Model)
 - 1. 程式結構與執行流程 (Structure of a Program and Execution Flow)
 - 2. 命名和綁定 (Naming and Binding)
 - 2.1 環境 (Environments)
 - 2.2 變數聲明 (let)
 - 2.3 變數賦值 (=)
 - 2.4 變數查找

- 2.5 函式閉包 (Function Closures)
- 3. 語義分析與變數解析 (Semantic Analysis and Variable Resolution)
 - 3.1 解析器 (Resolver) 的職責
 - 3.2 變數查找的優化
- 4. 例外 (Exceptions)
- 5. 程式執行方式 (Program Execution Methods)
 - 5.1 從檔案執行 (Executing from a File)
 - 5.2 互動式模式 (Interactive Prompt)
- 標準函式庫 (Standard Library)
- 錯誤處理 (Error Handling)

詞法結構 (Lexical Structure)

Treble Lin 程式碼由一系列 **Token** 組成, 這些 Token 是程式語言的最小有意義單位。

1. 識別字 (Identifiers)

識別字用於命名變數和函式。它們必須以字母或底線開頭, 後面可以跟隨字母、數字或底線。

- 範例: myVariable, calculate_sum, _temp

2. 關鍵字 (Keywords)

Treble Lin 保留了以下關鍵字:

- **and**: 邏輯與運算子
- **class**: 類別宣告 (目前未實作)
- **else**: if 語句的選擇性分支
- **false**: 布林值「假」
- **for**: 迴圈結構
- **fun**: 函式宣告
- **if**: 條件語句
- **nil**: 空值 (類似於其他語言的 null 或 None)
- **or**: 邏輯或運算子
- **return**: 從函式中返回值
- **super**: 呼叫父類別方法 (目前未實作)
- **this**: 引用當前實例 (目前未實作)
- **true**: 布林值「真」
- **let**: 變數宣告
- **while**: 迴圈結構
- **break**: 跳出當前迴圈
- **continue**: 跳過當前迴圈迭代
- **try**: 錯誤處理區塊的開始
- **catch**: 錯誤處理區塊, 用於捕獲 try 區塊中的錯誤
- **not**: 邏輯非運算子

3. 運算子和標點符號 (Operators and Punctuation)

Treble Lin 支援多種運算子和標點符號:

算術運算子 (Arithmetic Operators)

- **+**: 加法
- **-**: 減法
- *****: 乘法

- `/`: 除法

比較運算子 (Comparison Operators)

- `>`: 大於
- `>=`: 大於等於
- `<`: 小於
- `<=`: 小於等於

等值運算子 (Equality Operators)

- `==`: 等於
- `!=`: 不等於

邏輯運算子 (Logical Operators)

- `and`: 邏輯與
- `or`: 邏輯或
- `!, not`: 邏輯非

賦值運算子 (Assignment Operator)

- `=`: 賦值

其他標點符號 (Other Punctuation)

- `()`: 括號, 用於分組表達式或函式呼叫
- `{}`: 花括號, 用於定義程式碼區塊
- `,`: 逗號, 用於分隔函式參數或函式呼叫中的引數
- `.`: 點, 用於成員存取 (目前未實作, 但保留)
- `;`: 分號, 用於語句結束

4. 字面量 (Literals)

字面量是程式碼中直接表示固定值的表示方式。

- **數字 (Numbers)**: 支援整數和浮點數。
 - 範例: 123, 3.14, 0
 -
- **字串 (Strings)**: 用雙引號 " 包裹的字元序列。支援跳脫字元: `\n` (換行), `\t` (Tab), `\\` (反斜線), `\"` (雙引號)。
 - 範例: "Hello, World!", "這是包含\n換行的字串", "路徑: C:\\Users\\Name"
 -
- **布林值 (Booleans)**: true 和 false。
- **空值 (Nil)**: nil。

5. 註解 (Comments)

Treble Lin 支援兩種註解方式：

- 單行註解: 以 // 開頭, 直到行尾。
 - 範例: // 這是一行註解
- 多行註解: 以 /* 開頭, 以 */ 結束。
 - 範例:

```
/*  
 * 這是一個多行註解。  
 * 它可以跨越多行。  
 */
```

語法 (Syntax)

Treble Lin 的語法規則定義了如何組合 Token 以形成有效的程式碼結構。

1. 語句 (Statements)

語句是執行某些動作的指令。

1.1 表達式語句 (Expression Statements)

任何表達式後跟分號 ; 都可以作為語句。

- 語法: expression ;
- 範例:

```
1 + 2;  
"hello" + " world";
```

1.2 變數宣告 (Variable Declarations)

使用 let 關鍵字宣告變數, 可以選擇性地賦予初始值。

- 語法: let identifier [= expression] ;
- 範例:

```
let a;  
let b = 10;  
let message = "Hi";
```

1.3 程式碼區塊 (Block Statements)

使用花括號 {} 將多個語句組合在一個區塊中。區塊為變數創建新的作用域。

- 語法: { statement* }
- 範例:

```
{  
  let x = 1;  
  let y = x + 2;  
}
```

1.4 條件語句 (Conditional Statements)

使用 if 和可選的 else 關鍵字執行基於條件的程式碼。

- 語法: if (expression) statement [else statement]
- 範例:

```
let num = 10;  
if (num > 5) {  
  print("It's warm!");  
} else {  
  print("It's cool!");  
}
```

1.5 迴圈語句 (Loop Statements)

Treble Lin 支援 while 和 for 迴圈。

- **while** 迴圈: 當條件為真時重複執行程式碼。
 - 語法: while (expression) statement
 - 範例:

```
let i = 0;  
while (i < 3) {  
  print(i);  
  i = i + 1;  
}
```
- **for** 迴圈: 提供初始化、條件和遞增的綜合迴圈結構。
 - 語法: for ([initializer] ; [condition] ; [increment]) statement
 - 範例:

```
for (let i = 0; i < 3; i = i + 1) {  
  print(i);  
}
```

```
// 無限迴圈
for (;;) {
    print("無限循環!");
}
```

- **break** 語句: 立即終止最內層的迴圈。

- 語法: break ;
- 範例:

```
let i = 0;
while (true) {
    print(i);
    if (i == 2) {
        break; // 當 i 為 2 時跳出迴圈
    }
    i = i + 1;
}
```

- **continue** 語句: 跳過當前迴圈迭代中 continue 之後的程式碼, 並進入下一個迭代。

- 語法: continue ;
- 範例:

```
for (let i = 0; i < 5; i = i + 1) {
    if (i == 2) {
        continue; // 當 i 為 2 時跳過當前迭代
    }
    print(i);
}
// 輸出: 0, 1, 3, 4
```

1.6 函式宣告 (Function Declarations)

使用 fun 關鍵字宣告函式。

- 語法: fun identifier ([parameter (, parameter) *]) { statement* }
- 範例:

```
fun sayHello(name) {
    print("Hello, " + name + "!");
}
sayHello("Alice");
```

1.7 返回語句 (Return Statements)

在函式中使用 return 關鍵字返回一個值。

- 語法: return [expression] ;
- 範例:

```
fun add(a, b) {  
    return a + b;  
}  
let result = add(5, 3);  
print(result); // 輸出 8
```

1.8 錯誤處理 (Error Handling)

使用 try-catch 語句捕獲和處理運行時錯誤。

- 語法: try { statement* } catch (identifier) { statement* }
- 範例:

```
fun divide(a, b) {  
    // 由於 Treble Lin 沒有顯式拋出錯誤的機制,  
    // 這裡的錯誤將由解釋器內部的運行時錯誤觸發 (例如除以零)。  
    return a / b;  
}  
  
try {  
    let result = divide(10, 0); // 這會觸發除以零的運行時錯誤  
    print("結果: " + result);  
} catch (error) {  
    print("捕獲到錯誤: " + error);  
    // error 變數會包含錯誤訊息的字串表示, 例如 "Division by zero."  
}
```

2. 表達式 (Expressions)

表達式產生一個值。

2.1 賦值表達式 (Assignment Expressions)

將值賦給變數。

- 語法: identifier = expression
- 範例:

```
let x = 10;
```



```
x = x + 5; // x 現在是 15
```

2.2 邏輯表達式 (Logical Expressions)

Treble Lin 的邏輯運算子遵循短路求值 (**Short-circuit Evaluation**) 的原則，並且會根據值的「真值性 (Truthiness)」來判斷結果。

- **or**: 如果左側表達式求值為真，則返回左側的值；否則，求值右側表達式並返回其值。
 - 語法: expression or expression
 - 範例: true or false (結果為 true), 1 or 0 (結果為 1)
- **and**: 如果左側表達式求值為假，則返回左側的值；否則，求值右側表達式並返回其值。
 - 語法: expression and expression
 - 範例: true and false (結果為 false), 0 and "hello" (結果為 0)
- **not**: 對表達式進行邏輯非運算。
 - 語法: ! expression 或 not expression
 - 範例: not true (結果為 false), !0 (結果為 true)

2.3 等值表達式 (Equality Expressions)

比較兩個值是否相等或不相等。

- 語法: expression == expression 或 expression != expression
- 範例: 1 == 1 (結果為 true), "hello" != "world" (結果為 true), nil == nil (結果為 true)

2.4 比較表達式 (Comparison Expressions)

比較數字值。

- 語法: expression > expression, expression >= expression, expression < expression, expression <= expression
- 範例: 10 > 5 (結果為 true), 5 <= 5 (結果為 true)

2.5 算術表達式 (Arithmetic Expressions)

執行數學運算。

- 語法: expression + expression, expression - expression, expression * expression,

expression / expression

- 加法 (+) 的特殊行為：
 - 如果兩側都是數字，則執行數字加法。
 - 如果兩側都是字串，則執行字串連接。
 - 如果一側是字串，另一側是數字，則數字會被自動轉換為字串，然後執行字串連接。
- 範例: $(5 + 3) * 2$ (結果為 16), "Hello" + "World" (結果為 "HelloWorld"), "Count: " + 5 (結果為 "Count: 5")

2.6 一元表達式 (Unary Expressions)

對單個表達式執行運算。

- 減號 (Unary Minus): 將數字變為負數。
 - 語法: - expression
 - 範例: -5
- 邏輯非 (Logical Not): 對值進行邏輯非運算。
 - 語法: ! expression 或 not expression
 - 範例: not true (結果為 false), !0 (結果為 true)

2.7 函式呼叫表達式 (Call Expressions)

呼叫函式並傳遞引數。

- 語法: callee ([argument (, argument) *])
- 範例:

```
fun greet(name) {  
    print("Hello, " + name);  
}  
greet("Treble Lin User");
```

2.8 主要表達式 (Primary Expressions)

基本的表達式形式包括：

- 字面量: 數字、字串、布林值、nil。
 - 範例: 123, "text", true, nil

- 變數: 識別字, 表示一個變數。
 - 範例: myVariable
- 分組表達式: 用括號 () 包裹的表達式, 改變運算優先順序。
 - 語法: (expression)
 - 範例: (1 + 2) * 3

資料型別 (Data Types)

Treble Lin 是一種動態型別語言, 變數的型別在運行時確定。所有值都被視為 物件 (Objects), 它們具有其值和行為。主要支援的資料型別包括:

- 數字 (Numbers): 整數和浮點數。
- 字串 (Strings): 字元序列。
- 布林值 (Booleans): true 和 false。
- 空值 (Nil): 表示沒有值。
- 函式 (Functions): 一級公民, 可以像其他值一樣傳遞和賦值。

真值性 (Truthiness)

在 Treble Lin 中, 某些語法結構 (如 if 條件、while 條件、邏輯運算子) 需要判斷一個值的「真值性」。規則如下:

- nil 和 false 被視為假值 (falsy)。
- 所有其他值 (包括任何數字、非空字串、true 和函式) 都被視為真值 (truthy)。

執行模型 (Execution Model)

Treble Lin 程式的執行涉及到對語句的線性求值, 並通過環境 (Environments) 和解析器 (Resolver) 來管理變數的作用域。

1. 程式結構與執行流程 (Structure of a Program and Execution Flow)

Treble Lin 程式的執行流程通常分為以下階段:

1. 詞法分析 (Lexical Analysis): 原始碼被掃描並分解為一系列 Token。如果在該階段發現無法識別的字符或未終止的字串/註解, 會報告詞法錯誤。
2. 語法解析 (Parsing): Token 序列被解析以構建抽象語法樹 (Abstract Syntax Tree,

AST)。如果在該階段發現語法不符合語言規則，會報告解析錯誤。

3. **語義分析與變數解析 (Semantic Analysis and Variable Resolution)**: 在 AST 構建完成後，解釋器會遍歷 AST，確定每個變數引用的正確作用域，並將變數與其在環境中的「距離」綁定起來。這是在實際執行前的靜態分析步驟，用於捕獲作用域相關的錯誤（例如在同一作用域內重複聲明變數，或在初始化前讀取變數）。
4. **解釋執行 (Interpretation)**: 經過解析的 AST 會被解釋器逐語句執行。這個階段會處理運行時的行為，包括變數賦值、函式呼叫、運算求值等。

2. 命名和綁定 (Naming and Binding)

Treble Lin 使用詞法作用域 (**Lexical Scoping**) 來管理變數的命名和綁定。這意味著變數的解析是在程式碼編寫時(靜態地)決定的，而不是在運行時(動態地)決定的。

2.1 環境 (Environments)

環境是 Treble Lin 用來存儲和查找變數值的地方。每個程式碼區塊(例如函式體、if 語句的 then 或 else 分支、while 迴圈體、for 迴圈體以及獨立的花括號 {} 區塊)都會創建一個新的環境，該環境會指向其父環境。這形成了一個環境鏈，用於查找變數。

- **全域環境 (Global Environment)**: 程式執行的最外層環境，所有頂層聲明都在此環境中。
- **局部環境 (Local Environments)**: 當進入一個新的程式碼區塊(如函式呼叫或 {} 區塊)時，會創建一個新的局部環境，該環境以創建它的環境作為其父環境。

2.2 變數聲明 (let)

當使用 `let identifier` 聲明一個變數時，該變數會在當前環境中被定義。如果帶有初始值 (`let identifier = expression;`)，則該值會綁定到變數名上。

2.3 變數賦值 (=)

當執行賦值表達式 `identifier = expression` 時：

1. 解釋器會首先在解析階段確定的特定作用域層級查找 `identifier`。
2. 如果找到，則更新該變數的值。

3. 如果該變數是全域變數，則在全域環境中更新。如果是在局部作用域中，則在該作用域中更新。
4. 如果解析器未能綁定該變數(即它是一個未聲明的全域變數)，則會拋出運行時錯誤。

2.4 變數查找

當一個變數被引用時(例如在表達式中使用 `myVariable`)，解釋器會：

1. 使用解析階段計算出的變數「距離」來直接定位到正確的環境層級。
2. 從該環境層級獲取變數的值。
3. 如果解析器未能綁定該變數，或在運行時該變數在預期作用域中不存在，則會拋出運行時錯誤。

2.5 函式閉包 (Function Closures)

函式在定義時會「捕獲」其周圍的環境(稱為閉包環境)。這使得函式在被呼叫時，即使其定義時的外部作用域已經不存在，也能夠訪問到該環境中的變數。

- 範例：

```
fun outer() {  
  let x = "Hello";  
  fun inner() {  
    print(x); // inner 函式引用了外部函式的 x  
  }  
  return inner;  
}
```

```
let myFunc = outer();  
myFunc(); // 即使 outer 已經執行完畢，myFunc 仍然可以訪問到 x，輸出 "Hello"
```

在上述範例中，`inner` 函式形成了一個閉包，它記住了 `outer` 函式被呼叫時的環境，從而能夠在之後被呼叫時訪問 `x`。

3. 語義分析與變數解析 (Semantic Analysis and Variable Resolution)

在 Treble Lin 執行程式碼之前，會進行一個稱為解析 (**Resolution**) 的靜態分析階段。這個階段的目的是遍歷抽象語法樹 (AST)，並為每個變數引用確定其所指向的正確作用域層級，從而在運行時提供更快的變數查找，並在執行前捕獲某些作用域相關的錯誤。

3.1 解析器 (Resolver) 的職責

解析器負責：

- 跟蹤作用域：維護一個作用域棧 (scopes)，每個元素代表一個局部作用域。當進入新的程式碼區塊(如函式、if 或 {} 區塊)時，會開始一個新的作用域；當離開時，則結束作用域。
- 變數的聲明與定義：
 - 當聲明一個變數 (let identifier) 時，變數名會在當前作用域中被標記為已聲明但尚未定義。
 - 在變數的初始化表達式被解析後，變數名會被標記為已定義。這防止了在變數初始化完成之前訪問自身的情況(例如 let x = x + 1; 這樣的錯誤)，這種行為會在解析階段被捕獲並報告為錯誤。
- 解析局部變數：當解析器遇到一個變數引用時，它會從當前作用域開始，向外層作用域逐層查找該變數。一旦找到，解析器會計算該變數相對於當前環境的距離(即需要向上查找多少層環境才能找到它)，並將這個距離儲存起來，供運行時解釋器使用。
- 錯誤報告：在解析階段，如果發現作用域相關的錯誤(例如在同一作用域內重複聲明變數)，解析器會立即報告錯誤，防止這些問題進入運行時。

3.2 變數查找的優化

在 Treble Lin 中，變數查找的流程是：

1. 解析時：解析器 (Resolver) 靜態地確定每個變數引用相對於其定義的作用域「距離」。這個距離會儲存在 AST 節點上。
2. 運行時：解釋器 (Interpreter) 使用解析階段計算出的這個距離，直接跳轉到正確的環境層級來獲取或賦值變數，從而避免了運行時在整個環境鏈上進行動態搜索的開銷。對於全域變數，解析器不會儲存距離，解釋器會直接在全域環境中查找。

4. 例外 (Exceptions)

Treble Lin 使用 try-catch 語句來處理運行時錯誤。

- 當 try 區塊中的程式碼執行時發生運行時錯誤(例如除以零、類型不匹配)，執行流程將立即跳轉到最近的 catch 區塊。
- catch (identifier) 語句會將捕獲到的錯誤訊息(作為字串)綁定到 identifier 變數上，供 catch 區塊內部處理。
- 目前 Treble Lin 沒有提供顯式拋出錯誤(例如類似於 throw 或 raise 關鍵字)的語法，錯誤主要由解釋器在檢測到非法操作時自動產生。

5. 程式執行方式 (Program Execution Methods)

Treble Lin 程式可以通過以下兩種方式執行：

5.1 從檔案執行 (Executing from a File)

您可以使用命令行執行 Treble Lin 腳本檔案：

- 用法: `python main.py [script_filepath]`
- 範例: `python main.py my_script.treblelin`
- 程式將從指定檔案讀取並執行所有 Treble Lin 程式碼。

5.2 互動式模式 (Interactive Prompt)

如果沒有提供腳本檔案, Treble Lin 將進入互動式提示模式 (REPL - Read-Eval-Print Loop)。

- 用法: `python main.py`
- 提示符號: `>`
- 您可以在提示符後輸入 Treble Lin 程式碼, 按 Enter 執行。輸入 `exit()` 或按 Ctrl+D (EOF) 退出。按 Ctrl+C 中斷當前操作並退出。

標準函式庫 (Standard Library)

Treble Lin 目前只有一個內建函式：

`print()`

用於向標準輸出打印值。它支援多個引數, 會將它們轉換為字串並以空格分隔打印。它也支援類似 C 語言的格式化字串。

- 用法: `print(arg1, arg2, ...)`
- 格式化字串用法: `print("格式化字串 %s %d", string_value, number_value)`
 - `%s`: 替換為字串
 - `%d`: 替換為整數
 - `%f`: 替換為浮點數
- 範例:
`print("Hello", "Treble Lin!");` // 輸出: Hello Treble Lin!
`print(1 + 2);` // 輸出: 3
`print("數字: %d, 字串: %s", 123, "測試");` // 輸出: 數字: 123, 字串: 測試

錯誤處理 (Error Handling)

Treble Lin 在程式執行的不同階段都可能產生錯誤。一旦發生錯誤，程式的執行將會終止，並返回一個非零的退出碼。

- **詞法錯誤 (Lexical Errors):**
 - 原因: 原始碼中包含無法識別的字符，或字串/多行註解未正確終止。
 - 示例: `let x = @10;`
 - 退出碼: 65
- **語法解析錯誤 (Parse Errors):**
 - 原因: 程式碼不符合 Treble Lin 的語法規則(例如缺少括號、分號)。
 - 示例: `let x = (10 + 5;`
 - 退出碼: 65
- **語義解析錯誤 (Resolution Errors):**
 - 原因: 在靜態分析階段發現的變數作用域或綁定問題，例如：
 - 在同一作用域內重複聲明變數。
 - 在變數初始化完成之前嘗試讀取自身。
 - 示例: `{ let x = 1; let x = 2; }`
 - 退出碼: 65
- **運行時錯誤 (Runtime Errors):**
 - 原因: 程式在執行時遇到的錯誤，例如：
 - 對非數字操作數執行算術運算。
 - 除以零。
 - 呼叫函式時提供的引數數量不匹配。
 - 訪問未定義的變數(如果它未能被解析器靜態捕獲)。
 - 示例: `"hello" - 1;; 10 / 0;; print(undefinedVar);`
 - 退出碼: 70
- **檔案系統錯誤 (File System Errors):**
 - 原因: 在嘗試從檔案讀取程式碼時，指定檔案不存在。
 - 退出碼: 74

錯誤訊息會打印到標準錯誤輸出 (stderr)。